# Reviewer Pack — Minimal Local Index of Configuration Spaces vs ACC⁰ Circuit …

Entry 814b53775ed6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 09:02:52 UTC

## Minimal Local Index of Configuration Spaces vs ACC⁰ Circuit Depth for Sipser Functions

**Entry ID**: `814b53775ed6` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 09:02:52 UTC

### 1. Conjecture

**Field A** (mathematical branch): Configuration Space Theory **Field B** (complexity object): Complexity Theory: ACC⁰ Circuit Complexity (for Sipser Functions)

**Statement**:

> {'mathematical_statement': 'For every Sipser function f with n variables, the minimal local index of the configuration space associated with f is Θ(n^2).', 'counterexample_formulation': 'If there exists a Sipser function f with n variables such that the minimal local index of its configuration space is o(n^2), then the conjecture is refuted.'}

**Rationale (proposer's reasoning)**:

> {'connection_explanation': 'Configuration space theory provides a geometric perspective on computational processes, while ACC⁰ circuit complexity measures the size of circuits for Sipser functions. A connection between these fields could reveal underlying structural properties that affect computational hardness.', 'potential_structure_insight': 'Understanding the relationship between configuration spaces and ACC⁰ circuits could lead to novel insights into the structure of computation and potentially new algorithmic approaches.'}

**Taxonomy category**: `CONFIGURATION_SPACE_ACC0_CIRCUIT` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `6d2eb2673f7b6ddf`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> The conjecture is supported if the mean minimal local index of configuration spaces for 30 Sipser functions with varying numbers of variables, n, is within a factor of 3 from n^2.

## 3. Barrier filter (F1)

**Final**: PASS

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - `Minimal Local Index AND Configuration Space Theory AND ACC0 Circuit Depth` - `Sipser Functions AND Complexity Theory AND Configuration Space Theory` - `θ(n^2) AND Minimal Local Index AND Sipser Function Configuration Space`

**Top relevant hits considered**: - [http://arxiv.org/abs/1804.02985v1] Verification of Space Weather Forecasts issued by the Met Office Space Weather Operations Centre - [http://arxiv.org/abs/1208.1862v2] A transformation rule for the index of commuting operators - [http://arxiv.org/abs/2101.11121v3] Holomorphic representation of minimal surfaces in simply isotropic space - [http://arxiv.org/abs/hep-th/9707234v2] Variational Approach to Quantum Field Theory: Gaussian Approximation and the Perturbative Expansion around It - [http://arxiv.org/abs/1506.01945v1] On the error term in a Parseval type formula in the theory of Ramanujan expansions - [http://arxiv.org/abs/2210.12779v1] Islands of shape coexistence in proxy-SU(3) symmetry and in covariant density functional theory - [http://arxiv.org/abs/2311.02409v3] On free boundary minimal submanifolds in geodesic balls in $\mathbb{H}^n$ and $\mathbb{S}^n_+$ - [http://arxiv.org/abs/2508.09104v2] The stability index and Yau's conjecture for Carlotto-Schulz minimal hypertori

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def sipser_function(n, x):
    return sum(x[i] * (2 ** i) for i in range(n)) % 2 == 0

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    rank = 0
    for j in range(n):
        pivot_row = -1
        for i in range(rank, m):
            if A[i][j] != 0:
                pivot_row = i
                break
        if pivot_row == -1:
            continue
```

```
        A[pivot_row], A[rank] = A[rank], A[pivot_row]
        for i in range(m):
            if i != rank and A[i][j] != 0:
                factor = A[i][j] / A[rank][j]
                for k in range(n):
                    A[i][k] -= factor * A[rank][k]
        rank += 1
    return rank

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def homology_group_rank(n, seed):
    random.seed(seed)
    A = [[sipser_function(n, list(x[:i] + (j,) + x[i+1:])) for j in range(2)] for i in range(n)]
    rank = gaussian_elimination(A)
    return rank

def run_trial(seed: int) -> dict:
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        instances_tested = 30
        total_index = 0
        for _ in range(instances_tested):
            index = homology_group_rank(n, seed)
            if index < 0 or index > n * (n + 1) // 2:  # Upper bound on the rank of a matrix
                return {
                    "metric_name": "minimal_local_index",
                    "metric_value": None,
                    "instances_tested": instances_tested,
                    "conjecture_holds": False,
                    "counterexample": f"Invalid homology group rank {index} for n={n}"
                }
            total_index += index
        mean_index = Fraction(total_index, instances_tested)
        results.append({"n": n, "mean_index": mean_index})

    conjecture_holds = all(0.3 * n**2 <= result["mean_index"] <= 3 * n**2 for result in results)
    counterexample = "" if conjecture_holds else f"Failed for n={results[0]['n']}"

    return {
        "metric_name": "minimal_local_index",
        "metric_value": sum(result["mean_index"] for result in results) / len(results),
        "instances_tested": instances_tested * len(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
```

```python
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_122292c9.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_122292c9.py", line 63, in run_trial
    index = homology_group_rank(n, seed)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_122292c9.py", line 52, in homology_grou
    A = [[sipser_function(n, list(x[:i] + (j,) + x[i+1:])) for j in range(2)] for i in range(n)]
                                  ^
NameError: name 'x' is not defined
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> The test code crashed before producing data, which means that the pre-registered support condition could not be evaluated. | next: Investigate and fix the crash in the test code to evaluate the conjecture's support.

## 11. Audit log (LLM calls)

**Total LLM calls**: 10

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 11535 |
| 2 | propose | ollama_remote | glm4:latest | 0 | 0 | 10300 |
| 3 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 5528 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 4603 |
| 5 | novelty | ollama_remote | glm4:latest | 0 | 0 | 6923 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 11446 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 8655 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 11925 |
| 9 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 12150 |
| 10 | judge | ollama_remote | glm4:latest | 0 | 0 | 24640 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 107704 ms total latency. Provider mix: {'ollama_remote': 10}

*(full prompt+response transcripts available in `research/audit/814b53775ed6.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/814b53775ed6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/814b53775ed6.tar.gz` (if generated)